

## 第二章

### 梯度下降法的算法

#### 梯度下降法的变种，为什么这么变：

##### 一、基础原版：批量梯度下降 BGD (Batch GD)

做法：每轮用全部训练样本算全局梯度再更新参数

存在问题：大数据集计算极慢、单次迭代开销巨大

优化目标（衍生变种解决的痛点）：降低单次计算量、加速迭代

##### 二、变种 1：随机梯度下降 SGD P21

改进点：每次仅 1 个样本计算梯度更新

优化解决：BGD 全量计算太慢，大幅减少单次计算开销

遗留缺陷：梯度噪声大、参数更新震荡，收敛不稳定

##### 三、变种 2：小批量梯度下降 Mini-Batch SGD P21

改进点：折中方案，取一小批（batch）样本求平均梯度

优化解决：兼顾 BGD 稳定、SGD 速度，深度学习标准基础优化器

##### 四、变种 3：Momentum 动量梯度下降

改进点：累积历史梯度，引入惯性项平滑更新方向

优化解决：Mini-Batch/SGD 震荡严重、峡谷区域收敛慢；沿历史趋势加速下降

##### 五、变种 4：NAG 涅斯捷罗夫动量 (Nesterov Accelerated Gradient)

改进点：先预判下一步位置再算梯度（前瞻动量）

优化解决：普通动量会冲过极小值，NAG 提前修正，收敛更精准

##### 六、变种 5：AdaGrad 自适应梯度

改进点：每个参数独立自适应学习率，频繁更新参数步长大、稀疏参数步长大

优化解决：固定全局学习率无法适配不同参数梯度幅度差异

缺陷：梯度平方持续累加，后期学习率趋近 0，训练停滞

##### 七、变种 6：RMSprop

改进点：梯度平方做指数滑动平均，不无限累加

优化解决 AdaGrad 学习率持续衰减、后期不更新的问题

##### 八、变种 7：Adam (Adaptive Moment Estimation)

改进点：融合 Momentum 一阶动量 + RMSprop 二阶自适应，加偏差校正

优化解决：同时解决震荡、参数学习率不匹配、前期梯度估计偏差，深度学习最通用优化器

简版速记（考试背诵）

BGD：全样本，慢

SGD：单样本，快但震荡

Mini-Batch：折中，工业基础

Momentum：加惯性，消震荡、提速

NAG：前瞻动量，防止冲过最低点

AdaGrad：分参数自适应学习率，后期僵死

RMSprop：滑动平均梯度平方，修复 AdaGrad 衰减

Adam：动量 + 自适应结合，综合最优

为什么引入了深层的深度学习：单层速度太慢

典型的网络：是什么形式，实际网络的训练过程

分为正向传播：输入层、中间层、隐层、激活函数

反向传播：损失函数

某种损失函数搭配激活函数，出现什么问题，怎么解决

网络调优：三方面

拓扑结构

各个激活函数的特点和问题

变形函数的表达式：lru

损失函数：交叉熵损失函数在二分类情况下的特性

欠拟合，过拟合

过拟合为什么出现，怎么调：增多数据、调整模型结构变简单一些

泛化

针对过拟合的手段：参数惩罚、系数化、bending.....

训练策略：留一、k折：数据划分方式、各自优缺点：出现现象时换一种方式

## 第三章

卷积操作具体

和数学上卷积的区别

给一个特征图，卷积核，输出特征图大小

vgg lastnet典型结构

根据典型神经网络特性解决问题：大致画个框，接什么模块输出用什么，什么函数

卷积层输出公式：会算

过程拍是什么

卷积的步长，一次跳几格子

梯度下降法的原理，为什么可以找到参数，几种变体和特性

典型的神经网络：前一个变到后一个神经网络，后一个神经网络解决了什么问题：

LeNet（初代 CNN）→ AlexNet（P55-P58）

前一代缺陷：仅小手写数据集可用，Tanh 激活深层梯度消失，无正则，大图像分类效果差。

Alex 解决：ReLU 缓解梯度消失、Dropout + 数据增强抑制过拟合、双 GPU 并行，证明深层 CNN 在 ImageNet 可行。

AlexNet → VGG16/19（P59-P66）

前一代缺陷：卷积核尺寸杂乱、网络结构不规整，大卷积参数量大。

VGG 解决：统一堆叠 3x3 小卷积，同等感受野下参数更少，网络结构极简易复现，分层特征更丰富。

VGG → Inception（GoogLeNet 系列 P67-P77）

前一代缺陷：单纯堆叠卷积计算量大，单尺度特征适配差。

Inception 解决：多分支并行多尺度卷积；1x1 瓶颈卷积降参降计算；BN 加速收敛；辅助分类器缓解深层梯度消失。

普通深层网络（Plain）→ ResNet（P78-P83）

前一代致命问题：网络退化 —— 层数加深后训练 / 测试误差反而上升（BN 只能缓解梯度消失，无法解决退化）。

Res 解决：残差短路连接，网络只需学习残差而非完整映射，支持几十 / 上百层超深网络，精度大幅提升。

图像检测：RNN fastRNN

目标检测器的面积交并比怎么算

给定一个标签，一个目标区域，预测区域，算交并比

衡量指标：map

精度和召回率怎么算

精度真阳和假阳

RNN到fast RNN到fast RCNN的发展历史

yolo ssp

序列的因素

词源、token、embedding理解，为什么需要embedding

普通词嵌入和位置编码怎么做

循环神经网络的结构，有什么问题

后来提出的RSTM、GRU有什么优势

序列对序列结构

transformer

注意力机制怎么做

qkv怎么算，公式是什么

transformer典型结构

transformer到大模型的历史，和最终分类

哪些网络属于编码级解码级

哪些仅编码

哪些仅解码

为什么仅解码成为主流的原因

面向图像生成大模型：detection transformer：首次把什么特征融入目标检测，是里程碑

vip和swin框架

生成网络有对抗：分几种

扩散模型的正向扩散和反向过程

stable diffusion将传统的扩散模型优化效率，提高性能

clap VLA等等典型模型干了什么事，典型策略：把文字当标签，直接网上搜语料，自监督学习策略

智能体中部件：记忆，规划，工具是干什么的，有什么作用

量化：为什么要量化

数值量化分几种类型：分别怎么做

数值量化的优化方法：推理加速：分两种：训练后、训练中的流程、优缺点

kv量化的出现原因：大模型注意力机制中qkv运算频繁，kv量化很重要

chipcache作用，具体怎么做

## 第四章

张量tensor和普通的区别，互相怎么转化

tensor和cpu上的普通矩阵的区别，顺序不一样：传统cpu很难一个通道把数据聚到一起算

广播机制：给两个向量，判断是否满足广播条件，手算出广播结果

计算图：什么是静态图，什么是动态图，这两个编程框架分别支持什么图

两种图的优缺点

## 第五章

三个设计原则的哲学

包含哪四大模块

评分框架能帮助我们最大的工作量：自动指导

自动指导怎么一步步来，方法有什么缺点

给一个网络和上传参数：正向和反向的计算

计算图执行：

后端算子执行

实时编译为什么重要：冗余合并优化

一些优化机制：计算图级和算子图级别优化，给一个，判断是什么级别

单机优化：分布式架构

分布式架构分两种：

集合式怎么做：几种集合通信的原语怎么操作：给一个问题，判断我要用什么原语来完成

编译器优化：真正算子优化是对程序最底层的调优

分布式训练：划分策略分三种：数据并行、模型并行、混合并行：怎么操作

模型并行分两种

## 第六十七章

dip结构：两个最主要特点

1.取消数据缓存，直接io链接外存

2.添加便签，利用dma直接访问

dip优势：

一、存储架构带来核心优势（基于两大设计特点：无通用缓存 + 片上便笺 + DMA）  
省去 CPU/GPU 多级缓存硬件，减少芯片面积、降低硬件功耗，无缓存失效、颠簸开销  
片上 Scratchpad 便笺存储器搭配 DMA 批量搬运特征 / 权重，图像卷积规整访存完美适配，大幅降低 DRAM 外部访问次数，缓解内存墙瓶颈  
数据在片内高速流转，外部 IO 带宽利用率显著提升

二、计算单元硬件优势  
大规模并行 MAC 矩阵运算阵列，单周期完成大量乘加计算；CPU 为标量串行、GPU 并行度远低于 DLP，同等功耗下算力高出数倍  
原生支持低位宽定点 / 量化计算，适配神经网络 INT8 等低精度推理，计算、存储开销同步下降  
专为卷积、全连接、池化、激活等深度学习算子定制电路，无通用处理器冗余控制逻辑

三、控制电路优势  
仅简单循环计数控制流，无需复杂分支预测、乱序执行模块，控制电路面积积极小、功耗低  
深度学习任务无大量分支跳转，DLP 极简控制逻辑不会产生流水线气泡，执行效率稳定

四、综合性能优势  
高算力能效比：单位功耗下推理 / 训练吞吐量远高于 CPU、GPU，终端嵌入式 AI 场景优势极大  
低延迟：片上存储就近复用特征，减少片外数据往返延迟，适合实时图像识别、检测  
专用通用性平衡：指令集架构覆盖 CNN/RNN/Transformer 主流网络，专用硬件高能同时兼容各类深度学习模型  
稀疏运算原生支持，稀疏卷积、量化任务相比通用硬件速度提升明显

矩阵运算单元：

如何把多核的dip结合到一起

四个可以一个总线

多个只能嵌套

传统矩阵运算有什么问题：散出多，电路变慢，脉冲低：

散出多（扇出数量巨大）  
传统整块超大统一矩阵阵列（如 TPU 初代 256x256 PE 阵列），每个运算单元输出要连线到大量下游单元，信号线扇出极高，芯片布线极复杂，占用大量芯片面积，布线资源严重浪费。  
电路变慢（传输延迟高、时序难收敛）  
扇出多带来超长、杂乱金属走线，信号传输距离大幅增加，信号延迟变大；时钟同步难度上升，电路时序很难满足要求，芯片最高运行频率被压低，整体运算速度下降。  
脉冲利用率低（PE 闲置、算力浪费）  
图像 / 网络特征尺寸很难刚好匹配巨型阵列大小，大部分周期内大量 PE 无数据可算、处于空闲状态；时钟脉冲送到闲置单元完全无效，脉冲（时钟）利用率极低，硬件算力利用率差、能效很低。

## 第九章

优化分几种：

一、大模型优化分为四大类（文档 P26、P28）  
存储优化（核心 ZeRO 系列）  
计算优化：混合精度、FlashAttention、算子融合  
通信优化：梯度压缩、集合通信裁剪  
稳定性 / 容错优化

存储优化怎么调用存储空间，对大模型更好：ero

二、存储优化（ZeRO，你笔记 ero）：多级存储空间调用逻辑 P29-P31

1. 三层存储分层调度（显存→CPU 内存→NVMe 高速外存）
  - 1) 第一层：GPU HBM 显存（高速优先）  
仅保留当前计算分片的权重、梯度、激活、KV 缓存；前向 / 反向计算时 All-Gather 聚合完整层参数，计算完成立刻释放分片，不完整副本常驻显存，消除传统 DDP 每卡全量冗余。
  - 2) 第二层：CPU 主机内存（ZeRO-Offload）

闲置模型分片、优化器动量缓存异步卸载至 CPU DRAM；GPU 计算间隙通过 DMA 预取需要的分片，平衡显存压力，可支撑更大模型。

### 3) 第三层：NVMe 高速固态 (ZeRO-Infinity)

极少访问的底层模型分片、全局优化器状态存入 SSD；仅在切换高层时低速加载，极致突破单卡显存上限。

## 2. ZeRO 三阶段分片机制 (如何复用多级存储)

**ZeRO Stage1:** 仅分片优化器状态，仅占用少量 CPU 内存，显存节省中等

**ZeRO Stage2:** 分片优化器 + 梯度，CPU 卸载压力提升，显存大幅降低

**ZeRO Stage3:** 参数 / 梯度 / 优化器全部分片，配合 Offload/Infinity 三级存储协同，千亿级模型单集群可训练

## 3. 配套存储辅助手段

激活重计算 (计算换显存)、KV 分页缓存、模型量化，配合 ZeRO 共同降低存储占用。

## 稳定性优化：大模型故障怎么解决，找到对应策略解决

### 三、稳定性优化：大集群故障类型 + 对应解决策略 P37-P39

故障场景 1: GPU 宕机、节点掉线、硬件故障

方案 1: 定期检查点 Checkpoint

流程：每固定步数完整保存权重、优化器动量、数据集迭代状态、随机种子；故障重启加载最近存档继续训练。

缺点：故障间隔久会丢失大量迭代，存档读写开销大。

方案 2: 弹性训练 (Ooblock/Transom)

自动剔除故障节点、重构分布式并行拓扑，剩余节点继续训练，无需全局重启；动态增减算力，适配集群硬件波动。

故障场景 2: 内存软错误、梯度噪声、隐性数据损坏 (静默错误)

方案：冗余计算

低优先级资源同步重复计算同批次数据，对比两套梯度 / 损失结果，异常直接丢弃批次，避免坏数据污染模型。

故障场景 3: 通信超时、网络丢包

配套辅助稳定手段：梯度 1-bit 压缩、RDMA 高速通信、心跳监控节点状态，提前预警故障。